National University of Computer and Emerging Sciences 0498

Lahore Campus

COAL (EE2003)

Computer organization and assembly language

Date: November 5th 2024

Course Instructor(s)

AA,AA,SF,SI,SM

Sessional-II Exam

Total Time (Hrs): 1

Total Marks: 40

Total Questions: 2

40

23L-0700 Roll No

BCS-3K Section

Student Signature

Instructions : Attempt all questions. It is an open book exam only Assembly Language Programming Lecture Notes are allowed. Calculators are allowed. You can use rough sheets.

CLO #:2 Describe the working of important x86 assembly primitives, including arithmetic, branching, bit manipulation, addressing modes and interrupt handling.

Q1: [marks 4x2 + 2x2 + 2 + 2 +2 + 2 = 20]

(a) The segment and offset of interrupt service routine of int 77h are placed at: offset: 0x01DC segment: 0x01DE
(b) what is the total size of the IVT table? 1KB or 1024 bytes
(c) Which registers are changed by the iret instruction? CS, IP, SP, Flag
(d) Which interrupt is hooked by the instructions given on right side? $(64)_{16}$
	Code for part (d)
	mov ax, 0
	mov es, ax
	mov [es:400], mySub
	mov [es:402], cs

(e) Replace the following independent invalid instructions with a single instruction that has the same effect.

i)	mov ax, [ss:sp] add sp, 2	**Solution:** POP AX
ii)	sub sp,2 mov [ss:sp], ax	**Solution:** PUSH AX

(f) Write a code to swap two registers ax and bx without using temporary space (local variable) on stack or memory. You are only allowed to use **stack operations.**

Solution:

```
PUSH AX
PUSH BX
POP  AX
POP  BX
```

(g) What would be the value of SP after execution of the following code? Initial value of SP = 0xFFFE **Solution:** SP = 0xFFF8	Jmp start Routine: Ret start: Call routine Push ax Sub sp,4

(h) Complete the following code to place asterisk '*' character on the left diagonal of the screen.

1. Mov ax,0xb800
2. Mov es, ax
3. Mov ax,0x0742
   Mov si, 0 ?

20

4. Mov cx, 25
5. L1: Mov word [es: si] , ax
6. Add si, 166
7. Loop l1

; Since Rows are not completely divisible by Cols
; A diagonal starting from one corner to and ending at the other
; is not possible. Incrementing 3.2 ≈ 3 is the closest possible sol.

(i)

Suppose the following declarations have been made
str1: db 'FGHIJ'
str2: db 'ABCDE00000 '
Write instructions to move str1 to the end of str2, producing the string 'ABCDEFGHIJ' using string instructions
	Solution:
	JMP start
	...
	start:
	PUSH DS
	POP ES
	MOV CX, 5
	MOV SI, str1
	MOV DI, str2
	ADD DI, 5
		REP MOVSB
		MOV AX, 0x4C00
		INT 0x21

CLO #:2 Describe the working of important x86 assembly primitives, including arithmetic, branching, bit manipulation, addressing modes and interrupt handling.

Q2: Write a subroutine to swap the odd rows with the even ones in the video memory i.e. swap row 0 with row 1. Row 2 with row 3 and so on using string instructions. **[20 marks]**

Solution:

```
[org 0x0100]
      JMP start

buffer: times 80 dw 0                ; Temporary Buffer for Data Storage

swapRow:
        ; [BP + 8] Address of First Row
        ; [BP + 6] Address of Second Row
        ; [BP + 4] Address of Buffer

        PUSH BP
        MOV BP, SP

        PUSH DS
        PUSH ES
        PUSH AX
        PUSH CX
        PUSH SI
        PUSH DI

; Could've been optimized and shrinked
; by making a movRow subroutine
; and calling it thrice times with different parameters
```

```
PUSH DS
POP ES                          ; Pointing ES to DS
PUSH word 0xB800
POP DS                          ; Pointing DS to Video Memory

MOV CX, 80                      ; Word Count
MOV SI, [BP+8]                  ; Source is First Row
MOV DI, [BP+4]                  ; Destination is Buffer
REP MOVSW                       ; First Row → Buffer

MOV CX, 80                      ; Word Count
MOV SI, [BP+6]                  ; Source is Second Row
MOV DI, [BP+8]                  ; Destination is First Row

MOV AX, ES                      ; DS stored temporarily in AX/orig DS
PUSH DS
POP ES                          ; Pointing ES to Video Memory
REP MOVSW                       ; Second Row → First Row

MOV CX, 80                      ; Word Count
MOV SI, [BP+4]                  ; Source is Buffer
MOV DI, [BP+6]                  ; Destination is Second Row
MOV DS, AX                      ; DS pointing to original DS
REP MOVSW                       ; Buffer → Second Row

POP DI
POP SI
POP CX
POP AX
POP ES
POP DS

MOV SP, BP
POP BP

RET 6
```

swapAllRows:

```
        ; Accepts no Parameters
        PUSH AX
        PUSH CX                 ; Continued to Next Page
```

```
        MOV   CX, 12          ; Total Swaps
        MOV   AX, 0

swap_loop:
        PUSH  AX              ; [BP + 8] Address of First Row
        PUSH  AX              ; Incremented to next row
        ADD   AX, 160
        PUSH  AX              ; [BP + 6] Address of Second Row
        PUSH  word buffer     ; [BP+4] Address of Buffer
        CALL  swapRow

        ADD   AX, 160         ; Incremented to next row
        LOOP  swap_loop

        POP   CX
        POP   AX

        RET

start:
        CALL  swapAllRows

        MOV   AX, 0x4C00
        INT   0x21
```

20